



PRODUCT REQUIREMENTS DOCUMENT + TECHNICAL SPECIFICATION

RideScout

Full-Fledged iOS Ride Intelligence Platform

The definitive ride comparison platform for Apple users — every fare, every platform, every transit option, unified in a single beautifully crafted iOS experience. This document covers the complete product specification and backend technical architecture.

PART 1

Product Requirements Document

01 Executive Summary

RideScout is a premium iOS-native ride intelligence app that aggregates real-time fares, ETAs, and availability across all ride-hailing and public transit platforms. Users search once, see everything, book instantly — with full Apple ecosystem depth: Watch, CarPlay, Siri, Live Activities, Apple Wallet, and on-device ML. Built from ground up in SwiftUI, not a web wrapper.

800+

avg monthly overspend per commuter

4.3 min

wasted per trip switching apps

62M

iPhone users in India — TAM

02 Problem Space

Fragmented discovery

Commuters open 4–6 apps before every booking. Each requires separate location entry, independent loading, and shows no cross-platform context.

Invisible surge penalties

Surge can inflate fares 2–4x but users cannot compare surge states across platforms simultaneously or be alerted when one normalises.

Public transit blindness

Metro, BEST, BMTA, DTC options never surface alongside private rides. The cheapest 30 metro route is invisible when browsing 350 cabs.

Zero spend intelligence

No app aggregates cross-platform spend history. Users do not know their monthly ride spend or which platform saves them money per route.

03 Integrated Platforms

15+ services

Platform	Service Tiers
Uber	UberGo · Premier · XL · Moto · Auto · Connect
Ola	Mini · Sedan · Prime · SUV · Auto · Bike · Outstation

Rapido	Bike Taxi · Auto · Cab · Rapido Plus
InDrive	Bid-your-price rides · Intercity
Namma Yatri	Auto · Cab (Bengaluru, Kochi, Mysuru)
BluSmart	Electric cabs — Delhi, Bengaluru
Metro GTFS	DMRC · BMRCL · Chennai · Hyderabad · Mumbai
City Buses	BEST · BMTC · DTC · TSRTC via GTFS feeds
Shared Pools	Uber Pool · Ola Share · Quick Ride
Phase 2	Yulu · Bounce · Zoom Car · redBus · Meru · IRCTC

04

Target Users & Personas

AR	Arjun R., 26 — Daily Power Commuter Software Engineer · Bengaluru · iPhone 15 Pro Takes 2 rides daily, spends ₹4,000+/month. Expects Face ID on launch and Apple Pay at checkout. Watches surge alerts on Apple Watch during meetings. Needs cheapest option in under 10 seconds.
PS	Priya S., 35 — Safety-First Professional Marketing Director · Mumbai · iPhone 16 Pro Max Books rides for client meetings and late-night returns. Needs safety scores, live-share with family via iMessage, rated drivers. Uses Siri: "Hey Siri, book me a cab to BKC."
KT	Kavya T., 29 — Eco-Conscious Explorer UX Designer · Hyderabad · iPhone 15 + iPad Prefers Metro + e-bike combos, tracks CO2 saved. Shares split costs via Apple Pay Cash with flatmates. Uses iPad version for weekly commute planning sessions.
MK	Mohan K., 42 — Business Traveller Sales VP · Delhi · iPhone 16 + CarPlay Needs airport transfer options, corporate travel receipts, GST invoices in Apple Wallet. Uses Siri Shortcuts to auto-book his regular airport route.

05

Full Feature Set

Core — Ride Intelligence

Universal fare search

Enter origin + destination once. Every available ride type across 15+ platforms in a single ranked list. Auto-refreshes every 60s on active view.

Surge intelligence engine

Real-time surge multiplier per platform. Historical heatmap by hour and day. Predictive "surge-free window" alert on Watch when a platform drops below 1.2x.

Multi-modal route builder

Blended journeys: Metro + cab, bus + bike, walk + auto. Shows total cost, total time, CO2 vs full-cab alternative for every combination.

Deep-link booking handoff

One tap pre-fills pickup/drop in target app via Universal Links for Uber, Ola, Rapido, Namma Yatri. No retyping. Confirmed via receipt parsing.

Apple Ecosystem — Native Integrations

Built for the full Apple stack

Apple Watch app

Fare comparison on wrist. Surge alert complications. One-tap "book cheapest" action. Live Activity mirror for ride tracking.

CarPlay dashboard

Full fare search in CarPlay-native UI. Voice-first with Siri. Hands-free booking. Nearest pickup points while driving.

Live Activities + Dynamic Island

Post-booking Live Activity shows driver ETA, plate, real-time position. Dynamic Island compact view on iPhone 14 Pro+.

Apple Wallet passes

Metro QR tickets, booking confirmations, GST receipts as Wallet passes. Auto-updated with boarding details on trip day.

Siri & Shortcuts

Full SiriKit + App Shortcuts for iOS 17. "Hey Siri, cheapest ride to airport." User automations: "Every Friday at 5pm, search rides home."

Apple Pay & Cash

Premium subscription via Apple Pay. Group cost split as Apple Cash requests. One-tap GST invoice PDF download.

Calendar-aware suggestions

Reads Apple Calendar. 45 min before a meeting, surfaces fare comparison with live-traffic buffer built in.

Apple Maps integration

RideScout appears as ride option within Apple Maps. Tapping "Get Directions" shows fare estimates from RideScout inline.

Intelligence & Personalisation

Smart commute learning (Core ML)

Detects recurring routes on-device via Core ML. Proactively surfaces fare comparison for usual commute at typical departure time each morning.

Cross-platform spend analytics

Aggregates spend across all platforms via receipt parsing from Apple Mail + manual entry. Weekly and monthly breakdowns with Work/Personal category tags.

Carbon footprint tracker

CO2 per trip by vehicle type and distance. Cumulative eco savings from Metro and e-bike choices. Shareable monthly eco report card.

Driver trust scoring

Aggregated driver ratings across platforms. Flags unrated or below-3.8-star drivers. "Trusted driver" mode filters out low-rated options.

Live trip share via iMessage

Real-time trip progress, driver info, and ETA shared to any iMessage contact. iCloud-backed position relay. No account required for recipient.

Group cost split

Split fare estimate among up to 8 contacts. Apple Cash payment requests directly from split screen before the ride is booked.

06

Business Model

Tier	Price	Inclusions
Free	■0 / month	Unlimited fare searches, deep-link booking, basic surge view, Metro + bus routes, CO2 tracker. Revenue via affiliate commission ■8–25 per booked ride referral.
Scout Pro ★	■149 / month	Everything in Free + surge prediction alerts, spend analytics, calendar integration, driver trust filters, receipt export, 30s refresh rate, Apple Watch app, priority support.
Business	■499 / month	Everything in Pro + multi-user team dashboard, corporate policy engine (max fare caps), GST invoice aggregation, ERP integrations (Zoho, SAP), dedicated SLA.

Additional revenue streams

Platform-sponsored "Highlighted" badges (Uber/Ola pay for featured placement in non-surge results) · Ride insurance upsell (Digit, HDFC ERGO in-app policies) · Data insights API for urban planners (anonymised, aggregated, opt-in only) · White-label SDK licensing for travel super-apps.

07

Success Metrics & KPIs

Metric	Target	By	Priority
Monthly Active Users	1M MAU	Month 9	Critical
Search → deep-link conversion	>60%	Month 3	Critical
D30 retention rate	>42%	Month 6	Critical
App Store rating	≥4.6 stars	Month 4	High
Pro subscription conversion	>12% of MAU	Month 6	High
Avg. fare load time (p50)	<1.8s	Month 1	Critical
Watch app DAU / iPhone DAU	>25%	Month 8	Growth
Monthly revenue (all streams)	Rs.5Cr/mo	Month 15	Long-term
Crash-free session rate	>=99.5%	Month 1	Critical
Corporate accounts (Biz tier)	500 companies	Month 14	Long-term

08

Product Roadmap — 18 Months

Q1 2026	Foundation — Core App Launch 4 cities (Bengaluru, Mumbai, Delhi, Hyderabad). Uber + Ola + Rapido + Metro. Deep-link booking, Apple Pay subscription, Live Activities, Apple Wallet pass support, App Store launch with ratings campaign.
Q2 2026	Ecosystem Depth — Watch + CarPlay + Siri watchOS 10 app, CarPlay entitlement, SiriKit, App Shortcuts for iOS 17, calendar event ride suggestions, 8-city expansion (Chennai, Pune, Kolkata, Ahmedabad). BluSmart + Namma Yatri integration.
Q3 2026	Intelligence — Core ML + Analytics + iPad On-device commute pattern learning, predictive surge alerts, full cross-platform spend dashboard, CO2 report card, iPadOS multi-column layout, Yulu + Bounce + InDrive added.
Q4 2026	Business & International Expansion Corporate fleet policy engine, ERP integrations (Zoho, SAP), GST receipt Wallet pass, launch in Singapore and Kuala Lumpur with Grab and Gojek integration.

2027

Platform — Apple Maps + SDK

Native Apple Maps ride option powered by RideScout, open SDK for travel apps, inter-city rail + bus aggregation, real-time accessibility info, AirPods Pro voice-first mode.

09 Risks & Mitigations

Critical

Platform API access gated or revoked

Maintain three integration modes per platform: official affiliate API (preferred), structured scraping (fallback), manual fare tables every 4h (last resort). Proactively pitch Uber/Ola on revenue-sharing partnership for API access.

Critical

Apple Maps integration approval uncertain

Apply to Apple Maps partner programme Q1 2026. Build as deferred feature (Q4 roadmap) rather than dependency. App functions fully standalone without this integration.

High

Stale fares erode user trust

Display data freshness timestamp prominently. Animated shimmer when refreshing. Disclaimer on booking handoff: "Fares may differ at booking time." p95 refresh SLA of 90 seconds.

High

Low Pro tier conversion

Free tier deliberately excludes surge alerts and spend analytics — highest-value features for daily users. A/B test paywall placement: post-second-surge vs. day-7 nudge. 3-month free trial for new users.

Medium

Location data regulatory scrutiny

On-device location processing only via Core Location — never transmitted. DPDP Act 2023 + GDPR compliant. Quarterly privacy audits. Transparent data dashboard in Settings.

PART 2

Fare Aggregation Backend — Technical Specification

T01 System Overview

The RideScout backend is a horizontally scalable, event-driven microservices system deployed on AWS. Its primary responsibility is fetching, normalising, caching, and serving real-time fare data from 15+ ride platforms and GTFS transit feeds — all within a 1.8s p50 latency budget from client request to rendered result.

1.8s

p50 fare load latency target

60s

Redis cache TTL per route

99.7%

uptime SLA for aggregation layer

T02 Architecture Overview

Service	Responsibility
iOS Client (SwiftUI)	URLSession async/await → REST + WebSocket. On-device caching via Core Data. Optimistic UI with shimmer states during fetch. Background refresh via BGTaskScheduler.
API Gateway (AWS API GW + CloudFront)	TLS termination, rate limiting (100 req/min per device), JWT validation (Apple Sign-In tokens), request routing to fare-aggregation service.
Fare Aggregation Service (Node.js)	Primary microservice. Fans out parallel requests to all platform adapters. Normalises responses into canonical FareResult schema. Returns merged array sorted by price. Deployed on ECS Fargate.
Platform Adapter Layer	One adapter per platform (Uber, Ola, Rapido, etc.). Each adapter handles auth, request formation, response parsing, and error classification. Isolated failures — one adapter down does not affect others.
Redis Cache Cluster	AWS ElastiCache Redis. Key: SHA256(origin_lat, origin_lng, dest_lat, dest_lng, platform). TTL: 60s standard, 30s during detected surge events. Read-through pattern from aggregation service.

Kafka Event Bus	Real-time surge state change events published by adapters. Consumed by: surge prediction service, push notification service, analytics pipeline. Partitioned by city.
GTFS Transit Service	Separate Node.js service. Ingests GTFS static + GTFS-RT feeds for Metro and bus networks. Refreshes static data every 24h, real-time positions every 15s. Exposes unified transit fare + schedule endpoint.
Surge Prediction Service (Python)	Consumes Kafka surge events. XGBoost model trained on 6-month historical fare data per city-hour. Publishes surge_forecast events. Triggers APNs push via notification service.
Push Notification Service	Node.js + AWS SNS → APNs. Sends surge drop alerts and calendar-triggered ride suggestions. Respects user Focus mode state (stored in profile service).
Data Warehouse (Redshift + S3)	Anonymised, aggregated fare data pipeline via Kinesis Firehose → S3 → Redshift. Powers BI dashboard, surge model retraining, and (opt-in) data insights API for urban planners.

T03

Core API — Fare Search Endpoint

Request

```
POST /v1/fares/search
Authorization: Bearer
Content-Type: application/json

{
  "origin": { "lat": 12.9716, "lng": 77.5946 },
  "destination": { "lat": 12.9352, "lng": 77.6245 },
  "platforms": ["uber","ola","rapido","metro","bus"],
  "preferences": { "min_rating": 4.0, "max_fare": 500 },
  "client_ts": "2026-05-09T10:30:00+05:30"
}
```

Response

```
HTTP 200 OK

{
  "request_id": "req_01J2XK...",
  "fetch_at": "2026-05-09T10:30:01.243Z",
  "results": [
    {
      "platform": "rapido",
      "service_type": "bike_taxi",
      "fare": { "amount": 52, "currency": "INR", "surge_multiplier": 1.0 },
      "eta_minutes": 3,
      "vehicle": { "type": "bike", "seats": 1 },
      "driver_rating": 4.7,
      "deep_link": "rapido://book?pickup=12.9716,77.5946&drop;=12.9352,77.6245",
      "co2_grams": 18,
      "cache_age_seconds": 22
    },
    { "platform": "metro", "service_type": "transit", ... },
    { "platform": "uber", "service_type": "ubergo", ... }
  ],
  "surge_summary": { "uber": 1.4, "ola": 1.0, "rapido": 1.0 },
  "transit_options": [ { "mode": "metro+walk", "total_fare": 30, ... } ]
}
```

T04

Platform Adapter Specification

Adapter contract

Every platform adapter implements the PlatformAdapter interface. It receives a normalised SearchRequest and must return FareResult[] within 1.5s or throw a TimeoutError. The aggregation service wraps every adapter call in Promise.allSettled() — partial results are returned to the client with platform-level error states, never a 500.

Adapter integration modes (per platform)

Platform	Integration Mode	Auth	Data Freshness
Uber	Official Affiliate API	Bearer token, daily rotation	Real-time, 5s freshness
Ola	Partner API (v2)	OAuth 2.0, 1h expiry	Real-time, 10s freshness
Rapido	Affiliate deep-link API	API key, static	Fare estimate, ~30s freshness
InDrive	Screen-scraping (fallback)	Session cookie rotation	~60s freshness
Namma Yatri	Open API (ONDC)	No auth required	Real-time
BluSmart	Partner API (MoU required)	API key	Real-time

Metro GTFS	GTFS-RT feed (public)	No auth	15s real-time position
BEST/BM TC Bus	GTFS static + RT	No auth	15s real-time position

T05

Caching Strategy

Cache key design

SHA256(floor(lat*1000)/1000, floor(lng*1000)/1000, dest_lat, dest_lng, platform). Lat/lng floored to 3 decimal places (~110m grid) to maximise hit rate for nearby pickups.

Cache warming

Top 200 city routes pre-warmed every 45s by a scheduled Lambda. Reduces cold-start latency for common airport, station, and CBD routes to near-zero.

TTL policy

Standard: 60s. Surge detected (multiplier >1.2x): 30s. Off-peak hours (11pm–6am): 120s. GTFS transit data: 15s (RT positions), 24h (static schedules).

Eviction & memory

Redis maxmemory-policy: allkeys-lru. Per-city cluster to prevent cross-city eviction pressure. Alert threshold at 80% memory — triggers auto-scaling of cache nodes.

T06

Surge Detection & Prediction

Surge detection pipeline

Each platform adapter emits a SurgeEvent to Kafka when a fetched fare multiplier changes by more than 0.1x from the previous reading. The surge prediction service consumes these events, updates a rolling window, and re-scores the surge forecast for the next 30-minute window using the XGBoost model.

ML model specification

Algorithm

XGBoost regression. Trained per city + platform pair. Retrained weekly on Redshift fare history.

Features

Hour of day, day of week, local weather (OpenWeatherMap), local event calendar (Ticketmaster API), rolling 7-day surge average for that route cluster.

Output

Surge multiplier forecast for t+5, t+15, t+30 minutes. Confidence interval. "Surge-free by X:XX" string surfaced in Watch alert and Dynamic Island.

```
# Surge prediction event published to Kafka topic: surge.predictions
{
  "city": "bengaluru",
  "platform": "uber",
  "current_multiplier": 1.8,
  "forecast": [
    { "minutes": 5, "multiplier": 1.6, "confidence": 0.82 },
    { "minutes": 15, "multiplier": 1.2, "confidence": 0.71 },
    { "minutes": 30, "multiplier": 1.0, "confidence": 0.61 }
  ],
  "alert_trigger": "surge_free_at",
  "alert_message": "Uber likely surge-free in ~30 min"
}
```

T07 Push Notification Architecture

APNs delivery

Node.js notification service uses @parse/node-apn library. Sends via APNs HTTP/2 API. Production cert + sandbox cert managed in AWS Secrets Manager. Retry with exponential backoff on 503.

Notification types

surge_alert (time-sensitive, interruption-level: time-sensitive), calendar_ride_reminder (active), spend_weekly_summary (passive), live_activity_update (pushToStartToken for Live Activities on iOS 17+).

User preferences sync

Notification preferences stored in profile service (DynamoDB). Synced to device via CloudKit. Focus mode state read from device at notification send time via device-side filter in NotificationServiceExtension.

Rate limiting

Max 3 surge alerts per user per hour. Max 1 calendar reminder per event. Weekly summary: single push every Monday 9am local time. All limits enforced server-side in Redis.

T08 iOS Client Architecture

State management

SwiftUI + @Observable (iOS 17). Single AppStore actor for global state. Route-level ViewModels for local state. No third-party state library.

Networking layer

URLSession async/await. Custom FareAPIClient with automatic retry (3 attempts, exponential backoff). Response decoded via Codable. URLCache for offline fallback.

Core Data + CloudKit

User preferences, saved routes, spend history synced via NSPersistentCloudKitContainer. Auto-resolves merge conflicts with last-write-wins on non-critical fields.

Core ML integration

CommutePredictor.mlpack age compiled into app bundle. Inference runs on Neural Engine. Model updated quarterly via Core ML on-device personalisation pipeline (no data leaves device).

WidgetKit

3 widget families: systemSmall (cheapest fare), systemMedium (top-3 results), accessoryCircular (surge indicator for Lock Screen). All read from shared App Group Core Data store.

Live Activities

ActivityKit pushToStartToken registered on first launch. FareTrackingAttributes struct shared with Notification Service Extension. Dynamic Island compact/expanded views in SwiftUI.

T09 **Security & Privacy**

Auth & identity

Sign in with Apple as the sole auth method. No passwords stored. JWT validated server-side against Apple public keys (JWKS endpoint, cached 1h). User ID is Apple's anonymised identifier — not linked to email without explicit consent.

Location data handling

Core Location access: "When In Use" permission only, never "Always". Location sent in API request body (TLS 1.3), not stored server-side. Aggregation service processes coordinates and immediately discards after route computation.

Data minimisation

No device fingerprinting. No advertising identifier (ATT not requested). Analytics events are anonymised at source (no user ID in event payload). Server logs auto-purged after 7 days. DPDP Act 2023 compliant.

API security

All endpoints behind AWS WAF + Shield Standard. Rate limiting: 100 fare searches/min per JWT. Certificate pinning in iOS client for /v1/* endpoints. API keys for platform adapters in AWS Secrets Manager with 30-day rotation.

T10 **Infrastructure & Deployment**

AWS production infrastructure

Compute

ECS Fargate for all microservices. Auto-scaling on CPU >60% and p95 latency >800ms. Minimum 2 tasks per service across 2 AZs.

Database

RDS PostgreSQL (Multi-AZ) for user profiles and subscription state. DynamoDB for notification preferences (single-table design). Redshift for analytics warehouse.

Cache

ElastiCache Redis 7.x. Cluster mode enabled. Per-city shard to isolate eviction. Read replicas for fare-read traffic.

Messaging

Amazon MSK (managed Kafka). 3 brokers, replication factor 3. Topics: fare.fetched, surge.events, surge.predictions, notifications.queued.

CI/CD

GitHub Actions → ECR image build → ECS blue/green deployment via CodeDeploy. Automated rollback on CloudWatch alarm. Zero-downtime deploys.

Observability

Datadog APM for distributed tracing. CloudWatch for infra metrics. PagerDuty for on-call alerts. SLO dashboard: p50/p95/p99 fare latency, adapter error rate per platform, cache hit rate.

Disaster recovery

Multi-AZ by default. RTO target: 15 minutes. RPO: 1 hour (RDS automated snapshots every 1h). Annual DR drill required.

Cost targets

Infra cost <8% of revenue at scale. Fargate Spot instances for non-critical batch jobs (GTFS ingestion, model retraining).

APPROVALS & SIGN-OFF

Role	Responsibility	Signature	Date
Product Management	Full PRD reviewed and approved	_____	_____ —
Engineering Lead	Technical spec reviewed and feasible	_____	_____ —
Design Lead	UX requirements confirmed	_____	_____ —
Legal / Privacy	DPDP + App Store compliance confirmed	_____	_____ —
Security	Architecture security review passed	_____	_____ —

RideScout v2.0 Full PRD + Technical Spec	Confidential Do Not Distribute	Next Review: May 16, 2026 Status: Draft — Pending Approval
---	--------------------------------	--